At this point, the reorder test fails, even without the :js metadata tag, because the view is no longer contacting the server to do the reorder and reload the page.

Let's Talk About Drivers

To understand what Capybara is doing and what you need to do to make JavaScript integration testing work, you need to know about *drivers*. The driver is the part of Capybara that actually manages a web page as a series of DOM elements. The Capybara query language interacts with the driver, telling it to click a button or fill a form element, or just querying the driver to return DOM elements that match a selector and some other criteria.

Capybara's default driver is called Rack::Test. It's relatively simple and relatively fast. Rack::Test is written in Ruby and maintains its own DOM element tree. It's also dependent on Rack, which is the web stack that underlies nearly all Ruby web frameworks—not only Rails, but also Sinatra. Rack::Test has been great for our purposes so far, but it has some limitations. The most relevant of those for us is that Rack::Test doesn't have a JavaScript interpreter, and so ignores any JavaScript in the output. Once our code depends on JavaScript to do anything, Rack::Test is no longer useful.

Happily, if you're a web developer, there's about a 100% chance that you have multiple powerful JavaScript runtime interpreters on your computer in the form of web browsers. Capybara allows the use of other drivers that interact with the JavaScript interpreters designed for web browsers. Capybara still manages the query language, but hands over the queries and actions to the driver, which automates the actions using the full web rendering engine, including JavaScript.

Until 2017, I'd have said the three most commonly used JavaScript drivers for Capybara were Selenium, capybara-webkit, and Poltergeist. Selenium is the oldest, Capybara ships with support for it, and it differs from the other two in that it is a toolkit designed to automate a browser that is already installed on your computer and that Capybara can be used to control. That is to say that by default, when you run Selenium tests, you'll see a browser window open and the interactions actually play out in the window. While this has a certain comforting concreteness to it, it's also kind of slow. Poltergiest and capybara-webkit are both tools that extract the WebKit rendering engine used in the Safari browser into a standalone tool that is designed to run "headless," meaning without a window. While capybara-webkit and Poltergeist typically run faster than Selenium, they are both somewhat tricky to install and notoriously a little flaky to run.